

Documentación Complementaria del Vehículo Autónomo

1. Manual de conexión

Alimentación:

- Conectar el **portapilas 18650** al módulo **driver de motores** (pines + y - de alimentación).
- El **Arduino Nano** puede recibir 5V desde el regulador del driver o desde la salida de un módulo reductor de voltaje.
- El **servo motor** también debe conectarse a 5V (no directamente a 7.4V).

Motores DC (tracción):

- Conectar los dos motores DC a las salidas **OUT1–OUT2** y **OUT3–OUT4** del driver de motores.
- Conectar las entradas **IN1, IN2, IN3, IN4** del driver a pines digitales del Arduino (ej. 4, 5, 6, 7).
- Conectar pines **ENA y ENB** del driver a pines PWM del Arduino (ej. 9 y 10) para controlar velocidad.

Sensor ultrasónico HC-SR04:

- VCC → 5V Arduino.
- GND → GND Arduino.
- TRIG → pin digital 8 Arduino.
- ECHO → pin digital 7 Arduino.

Servo motor:

- VCC → 5V Arduino/regulador.
- GND → GND común.
- Señal → pin digital 3 Arduino (PWM).

2. Archivos de código (.ino)

```
#include <Arduino.h>

#include <Servo.h>

// Sensor lateral izquierdo

const int trig2 = A3; // Trig del sensor izquierdo

const int echo2 = A4; // Echo del sensor izquierdo

// Pines del L298N

const int IN1 = 8;
```

```

const int IN2 = 9;

// Servo en pin 4
Servo direccion;

const int servoPin = 4;

// Variables de distancia
long duracion1;
int distancia1;

// Estado del servo
bool servoGirado = false;

bool bloqueoSensor = false; // evita giros en bucle

unsigned long tiempoGiro = 0;

// Contador para controlar retroceso a la tercera detección
int contadorDetecciones = 0;

void setup() {
    pinMode(trig2, OUTPUT);
    pinMode(echo2, INPUT);
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);

    // Servo
    direccion.attach(servoPin);
    direccion.write(90); // arranca recto
    Serial.begin(9600);

    // Espera 3 segundos antes de encender motor
    delay(3000);

    // Motor adelante siempre
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
}

// Función para medir distancia del sensor
int medirDistancia(int trigPin, int echoPin) {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    long duracion = pulseIn(echoPin, HIGH, 20000); // timeout 20ms
    if (duracion == 0) return 999; // sin lectura válida
    return duracion * 0.034 / 2; // distancia en cm
}

```

```

}

void loop() {
    // Leer sensor

    distancia1 = medirDistancia(trig2, echo2);

    // --- Detección válida para girar ---

    if (!servoGirado && !bloqueoSensor && distancia1 > 60) {
        contadorDetecciones++; // aumenta cada vez que hay una detección válida

        if (contadorDetecciones == 3) {
            // --- RETROCEDER 1 segundo ---

            digitalWrite(IN1, LOW);

            digitalWrite(IN2, HIGH); // reversa

            delay(1000);

            // --- DETENER ---

            digitalWrite(IN1, LOW);

            digitalWrite(IN2, LOW);

            delay(100);

            // Reiniciar contador después del retroceso

            contadorDetecciones = 0;
        }

        // --- GIRAR servo a la DERECHA ---

        direccion.write(55); // gira a la derecha

        servoGirado = true;

        tiempoGiro = millis();

        bloqueoSensor = true;

        // Reanudar avance

        digitalWrite(IN1, HIGH);

        digitalWrite(IN2, LOW);
    }

    // Si ya giró y pasaron 0.5 segundos, volver al centro

    if (servoGirado && (millis() - tiempoGiro >= 500)) {
        direccion.write(90); // regresa recto

        servoGirado = false;
    }

    // Liberar bloqueo si vuelve a detectar pared cerca

    if (distancia1 <= 60) {
        bloqueoSensor = false;
    }

    // Debug en monitor serie

```

```

Serial.print("Distancia: ");

Serial.print(distancia1);

Serial.print(" cm | Servo: ");

Serial.print(servoGirado ? 55 : 90);

Serial.print(" | Detecciones: ");

Serial.println(contadorDetecciones);

delay(50); // frecuencia de actualización
}

```

3. Esquema eléctrico

- Arduino Nano en el centro.
- Driver de motores conectado a los motores DC y a las baterías.
- Sensor ultrasónico al frente (TRIG/ECHO a pines digitales).
- Servo motor conectado a pin PWM.
- GND de todos los módulos unido en común.

4. Diagrama de flujo del Código Arduino

Este diagrama representa las funciones principales: **Inicialización (setup)** y el **Bucle Principal (loop)**.

1. Inicialización (setup)

El proceso de inicio del robot, donde se configuran pines y se establecen los estados iniciales.

Fragmento de código

```

graph TD
    A[Inicio] --> B{Configurar pines de entrada/salida: Trig/Echo, Motores IN1/IN2};
    B --> C[Adjuntar Servo a pin 4];
    C --> D[Servo a 90 grados (Recto)];
    D --> E[Inicializar Comunicación Serial (9600)];
    E --> F[Esperar 3000 ms (3 segundos)];
    F --> G[Motor Adelante: IN1=HIGH, IN2=LOW];
    G --> H[Fin Setup];

```

2. Bucle Principal (loop)

Esta es la lógica de control principal que se repite continuamente.

Fragmento de código

```

graph TD
    A[Inicio Loop] --> B{medirDistancia(trig2, echo2)};
    B --> C[distancia1];

```

```

C --> D{¿servoGirado? = FALSO?};
D -- SI --> E{¿bloqueoSensor? = FALSO?};
D -- NO --> H;

E -- SI --> F{¿distancial > 60 cm?};
E -- NO --> H;

F -- SI --> G[contadorDetecciones++];
F -- NO --> H;

G --> I{¿contadorDetecciones == 3?};

I -- SI --> J[Motor Reversa: IN1=LOW, IN2=HIGH];
J --> K[Esperar 1000 ms];
K --> L[Motor Detener: IN1=LOW, IN2=LOW];
L --> M[Esperar 100 ms];
M --> N[contadorDetecciones = 0];
N --> O;

I -- NO --> O;

O[Servo a 55 grados (Derecha)];
O --> P[servoGirado = VERDADERO];
P --> Q[tiempoGiro = millis()];
Q --> R[bloqueoSensor = VERDADERO];
R --> S[Motor Adelante: IN1=HIGH, IN2=LOW];
S --> H;

H[Check Giro];

H --> T{¿servoGirado? = VERDADERO?};
T -- SI --> U{¿millis() - tiempoGiro >= 500 ms?};
T -- NO --> V;

U -- SI --> W[Servo a 90 grados (Recto)];
U -- SI --> X[servoGirado = FALSO];
X --> V;
U -- NO --> V;

V[Check Desbloqueo];

V --> Y{¿distancial <= 60 cm?};
Y -- SI --> Z[bloqueoSensor = FALSO];
Y -- NO --> Z_no;

Z --> Z_no;

Z_no[Imprimir Debug en Serial];
Z_no --> AA[Esperar 50 ms];
AA --> A;

```

Explicación clara del código (en español)

A continuación te doy una lectura paso a paso de este diagrama de flujo en formato Mermaid que parece controlar un robot con un sensor de distancia, un servomotor y un motor de giro. Desgloso cada parte, qué significa y cómo se comporta.

◇ Visión general

- El diagrama describe un bucle principal que empieza en “Inicio Loop”.
- Se mide la distancia usando un par de pines (trig2, echo2) y se obtiene una `distancia1`.
- El flujo maneja dos estados principales: giro del servo y detección de bloqueo del sensor, para decidir si el motor debe avanzar, retroceder o detenerse.
- Llega a un punto de impresión de debug y espera, cerrando el bucle para volver a empezar.

◇ Detalle por secciones

Inicio del bucle y medición

- **A:** Inicio del loop.
- **B:** El bloque condicional mide la distancia con `medirDistancia(trig2, echo2)`.
- **C:** Se obtiene `distancia1` (valor medido).

Decisión sobre si el servo está girado

- **D:** Condición: `¿servoGirado? = FALSO?`
 - Si **SI**: se pasa a la verificación de bloqueo del sensor (**E**).
 - Si **NO**: se va directamente a **H** (Check Giro) para recuperar el flujo.

Verificación de bloqueo del sensor

- **E:** Condición: `¿bloqueoSensor? = FALSO?`
 - Si **SI**: se evalúa la distancia en el bloque **F**.
 - Si **NO**: se va a **H** (Check Giro).

Comprobación de distancia cuando el sensor no está bloqueado

- **F:** Condición: `¿distancia1 > 60 cm?`
 - Si **SI**: se incrementa el contador de detecciones en **G**.
 - Si **NO**: se va a **H**.

Contador de detecciones y acción de giro/avance

- **G:** `contadorDetecciones++` (incrementa cuando la distancia es mayor a 60 cm).
- **I:** Condición: `¿contadorDetecciones == 3?`
 - Si **SI**:
 - **J:** Motor Reversa: `IN1=LOW, IN2=HIGH` (marcha atrás).
 - **K:** Esperar 1000 ms.
 - **L:** Motor Detener: `IN1=LOW, IN2=LOW`.
 - **M:** Esperar 100 ms.
 - **N:** `contadorDetecciones = 0` (reinicia el contador).
 - **O:** continúa.
 - Si **NO**: va a **O** para avanzar con giro.

Giro del servo cuando se detectan varias distancias largas

- **O:** Servo a 55 grados (Derecha) (gira el servo a la derecha).
- **P:** servoGirado = VERDADERO (marca que el servo ya ha girado).
- **Q:** tiempoGiro = millis() (captura el tiempo actual).
- **R:** bloqueoSensor = VERDADERO (bloquea sensores temporalmente).
- **S:** Motor Adelante: IN1=HIGH, IN2=LOW (mueve el robot hacia adelante).
- **H:** bloque "Check Giro" (vuelve a la comprobación de giro).

Check Giro

- **H:** Check Giro.
- **T:** Condición: ¿servoGirado? = VERDADERO?
 - Si **SI**: va a **U** para comprobar cuánto tiempo desde el giro.
 - Si **NO**: va a **V** (Check Desbloqueo).

Tiempo desde el giro para volver a la posición recta

- **U:** Condición: ¿millis() - tiempoGiro >= 500 ms?
 - Si **SI**: va a **W** para volver a la posición recta y a **X** para desbloquear.
 - Si **NO**: va a **V**.
- Si se cumple el tiempo:
 - **W:** Servo a 90 grados (Recto) (retorna el servo a la posición recta).
 - **X:** servoGirado = FALSO (marca que ya no está girado).
 - Luego va a **V**.
- **V:** Check Desbloqueo.

Desbloqueo y impresión de debug

- **V:** Check Desbloqueo.
- **Y:** Condición: ¿distancial <= 60 cm?
 - Si **SI**: bloqueoSensor = FALSO (desbloquea el sensor).
 - Si **NO**: va a **Z_no**.
- **Z_no:** Bloque que imprime en Serial para debug.
 - **AA:** Esperar 50 ms.
 - **A:** Vuelve al inicio del bucle.

◇ Flujo clave y estados

- Estados principales:
 - servoGirado (booleano): indica si el servo se ha girado.
 - bloqueoSensor (booleano): si el sensor está bloqueado para evitar lecturas bruscas.
 - contadorDetecciones (contador): acumula lecturas positivas de distancia mayor a 60 cm.
- Acciones de movimiento:
 - Motor Adelante (IN1=HIGH, IN2=LOW): avanza.
 - Motor Reversa (IN1=LOW, IN2=HIGH): retrocede.
 - Motor Detener (IN1=LOW, IN2=LOW): detiene.
- Acciones del servo:

- o Servo a 55 grados y Servo a 90 grados: bloquean y liberan el giro para posicionamiento.
- Temporización:
 - o Esperas de 1000 ms y 100 ms al retroceder.
 - o Espera de 500 ms para volver a recto después de girar.
 - o Espera de 50 ms para debug.

¿Qué intenta lograr este diagrama?

- Explorar mediante un sensor de distancia si hay obstáculos cercanos.
- Usar un servo para orientar el sensor (giro a la derecha y volver al centro).
- Si se detectan varias lecturas largas (distancia > 60 cm) en fila (contador igual a 3), el robot retrocede un momento para evitar un posible colisión.
- Si el gestor de giro indica que ya se giró, se intenta recuperar el estado recto tras un breve periodo.
- Mantener un bucle de lectura y depuración por puerto serie para monitorear en tiempo real.

Pequeñas notas de implementación

- Asegúrate de que las condiciones sean consistentes con tu lenguaje de ejecución (p. ej., Arduino/C++).
- Verifica que las variables booleanas (`servoGirado`, `bloqueoSensor`) y el contador (`contadorDetecciones`) estén inicializadas:
 - o Ejemplos: `servoGirado = false; bloqueoSensor = false; contadorDetecciones = 0;`
- Verifica que las funciones de medición y control del motor/servo existan y reciban los pines correctos.
- Si quieres simplificar, puedes:
 - o Reducir el número de estados y usar un simple FSM (finite state machine) con estados como: `BUSCAR_DISTANCIA`, `GIRA`, `AVANZAR`, `DETENER`, `REGRESAR`, `DEBUG`.

5. Protocolos de comunicación

- **UART (Serial):** Arduino ↔ PC (para depuración y envío de datos).
- **PWM (Pulse Width Modulation):** usado en servo motor y en control de velocidad de motores DC.
- **Digital Trigger/Echo:** protocolo básico para el sensor HC-SR04 (pulso de salida → espera rebote → mide tiempo).